

АГРЕГИРАЩИ ФУНКЦИИ – MIN(), MAX(), AVG(), SUM(), COUNT()

- MIN() – minimal
- MAX() – maximal
- AVG() – average – средна
- SUM() - сума
- COUNT() – брой

Count(*) - връща броя на всички редове.

EXPRESSION - без NULL.

DISTINCT – без повторения и без NULL.

Нека преговорим по познатия начин, като вече свикваме да проверяваме броя на засегнатите редове в резултата:

```
USE geography;
```

```
-- Намерете имената и височините на всички планини - резултат 48 реда
```

```
SELECT peak_name, elevation  
FROM peaks;
```

peak_name	elevation
Aconcagua	6962
Botev	2376
Carstensz Pyramid	4884
Damavand	5610
Dykh-Tau	5205
Elbrus	5642
Everest	8848
Julianatop	4760
K2	8611

39 11:21:11 SELECT peak_name, elevation FROM peaks LIMIT 0, 1000

48 row(s) returned

Нека проверим с функцията за преброяване:

```
-- Колко е броят на всички върхове?
```

```
SELECT COUNT(*) FROM peaks; -- 1 ред резултат за 48 записа
```

Result Grid
COUNT(*)
48

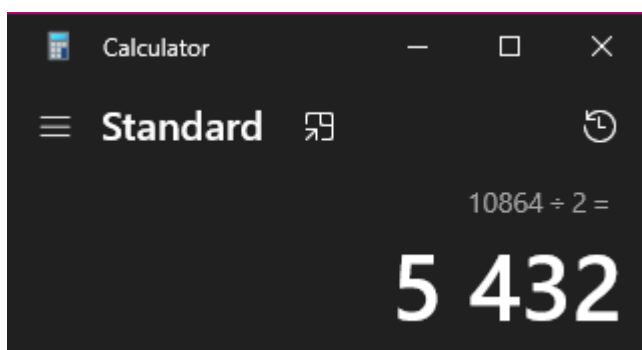
40 11:23:34 SELECT COUNT(*) FROM peaks LIMIT 0, 1000

1 row(s) returned

Нека тестваме агрегиращите функции и им направим проверки:

```
-- Агрегиращи функции
SELECT MAX(elevation) FROM peaks; -- 8848
SELECT MIN(elevation) FROM peaks; -- 2016
SELECT AVG(elevation) FROM peaks; -- 4891.2708
SELECT SUM(elevation) FROM peaks; -- 234781
SELECT SUM(elevation) / 48 FROM peaks; -- проверка на AVG
SELECT SUM(elevation) / COUNT(elevation) FROM peaks; -- проверка 2 на AVG
```

Т.е. уверихме се, че функцията AVG наистина осреднява сумата на всички стойности като ги събира и дели на техния брой, а не просто максималната и минималната стойност $(8848+2016)/2$, но една допълнителна проверка никога не е излишна.



Аналогично можем да намерим най-дълга / къса река река или най-висока / ниска заплата и моделираме задачи за тях със средни стойности, брой и суми.

Нека намерим най-високия връх и неговото име както сме го правили досега, припомням, че ORDER BY може да има 2 подредби:

ASC ascending – от малко към голямо (а -> я, а -> z, 0 -> 100) – тя е по подразбиране, ако не пише нищо, в нарастващ ред

DESC descending – от голямо към малко – в обратен ред от 100 -> 0, я -> а.

За да намерим най-голямото, подреждаме от голямо към малко и срязваме 1 резултат:

```
-- Намерете името и височината на най-високия връх
SELECT peak_name, elevation FROM peaks
ORDER BY elevation DESC
LIMIT 1;
```

peak_name	elevation
Everest	8848

Ако се опитаме да използваме същия метод с агрегиращи функции, обаче, ще хвърли грешка:

```
-- Име и височина на най-висок връх с MAX() - грешка:
SELECT peak_name, MAX(elevation) FROM peaks;
```

48 11:35:40 SELECT... Error Code: 1140. In aggregated query without GROUP BY, expression #1 of SELECT list contains nonaggregated column 'geography.peaks.peak_name', this is incompatible with.

Т.е. добавянето на 1 име на колона без групиране, дава несъвместимост.

Нека групираме по планини чрез тяхното id:

```
-- Име и височина на най-висок връх с MAX + GROUP BY - грешка:
SELECT peak_name, MAX(elevation) FROM peaks
GROUP BY mountain_id;
```

Отново хвърля същата грешка, т.к. името на планината не е съвместимо с критерия за групиране. Ами, нека го направим съвместим (уеднаквим):

```
-- Най-високият връх за всяка планина с MAX + GROUP BY:
SELECT mountain_id, MAX(elevation) FROM peaks
GROUP BY mountain_id;
```

Т.е. единствената разрешена колона до агрегиращите функции е тази, която съвпада с групиращия критерий!

Чрез този критерий за групиране получихме най-високите върхове от всяка планина.

mountain_id	MAX(elevation)
1	6194
2	5610
3	6962
4	2376
5	5642
7	4897
8	4285

```
55 11:59:24 SELECT mountain_id, MAX(elevation) FROM peaks GROUP BY mountain_id LIMIT 0, 1000 22 row(s) returned
```

Забелязваме, че редовете са по-малко 22 от общо 48, т.е. има достоверност.

Т.е. основната функция на агрегиращите функции е те да се използват самостоятелно, извеждайки MAX, MIN, AVG, SUM, COUNT от дадена група.

```
-- Най-високият връх за всяка планина с MAX + GROUP BY -- 22 реда:
SELECT MAX(elevation) FROM peaks
GROUP BY mountain_id;
```

Кое то вече е стандартния запис:

MAX(elevation)
6194
5610
6962
2376
5642
4897
4285
8848
4760

Съответно можем да подредим стойностите по височина и да зададем псевдоним:

```
-- Най-високият връх за всяка планина, подредени по височина:
SELECT MAX(elevation) AS Elevation
FROM peaks
GROUP BY mountain_id
ORDER BY elevation DESC;
```

	Elevation
▶	8848
	8611
	6962
	6194
	5959
	5895
	5642
	5636
	5610

Този път в намаляващ ред, отново 22 записа.

Т.е., ако съпоставим двата метода – с вградената MAX имаме точно 1 резултат, ограничена е откъм брой колони в резултата, макар привидно да изглежда по-лесна и логична.

Същият код, написан с LIMIT дава много по-голяма гъвкавост, т.к. могат да се върнат повече редове резултат по избор (например 3-те най-малки) и нямаме ограничения за включените колони, които се изброяват:

```
-- Най-високият връх за всяка планина, подредени по височина:
SELECT MAX(elevation) AS Elevation    вградена (ограничена)
FROM peaks                            функция
GROUP BY mountain_id
ORDER BY elevation DESC;
||| два еквивалентни кода
-- Извод: Limit дава много по-голяма гъвкавост
-- включват се повече колони и може да върне повече стойности:
SELECT elevation AS Elevation        по-гъвкава
FROM peaks
ORDER BY elevation DESC
LIMIT 1;
```

Нека видим как правилно да включим и имената – чрез ПОДЗАЯВКА, която можем да свиваме и разгъваме:

```

-- Името и височината на най-високите върхове за всяка планина,
-- подредени по височина с MAX + GROUP BY:
SELECT peak_name, elevation
FROM peaks
+ WHERE elevation IN присвояване
(
SELECT Max(elevation)
FROM peaks подзаявка
GROUP BY mountain_id
)
ORDER BY elevation DESC; последно подреждане

```

Вместо IN, в някои версии може да се използва = ANY, = SOME или просто =.

За височина от дадената група присвояваме максималната стойност от подзаявката. Подзаявката може да бъде свивана и разгъвана с (+) -> (-).

Ако има подреждане на резултата, то винаги остава последно извън подзаявката.

```

-- Нека се върнем към познатия начин и намерим отново
-- височината на най-високия връх:
SELECT elevation
FROM peaks
ORDER BY elevation DESC
LIMIT 1;

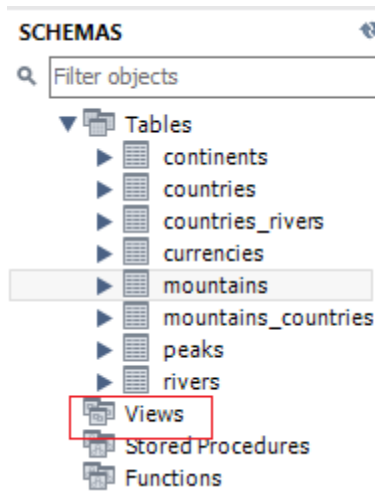
-- Намерете най-високите върхове от всяка планина
-- по друг начин - order by + limit
SELECT peak_name, elevation
FROM peaks AS p = виртуално копие
WHERE elevation =
(
SELECT elevation
FROM peaks
WHERE mountain_id = p.mountain_id -- цикъл
ORDER BY elevation DESC
LIMIT 1
)
ORDER BY elevation DESC;

```

Реално, познатият код за намиране на най-високият връх го ограждаме в скоби и вкарваме като подзаявка. След това, присвояваме върнатата стойност на намерената височина, която играе роля на своеобразна променлива по подобен на C# начин. За да има изброяване на група, изреждане от дадена категория, вкарваме реда WHERE, където сме избрали да се изреждат последователно планините чрез тяхното ID. Ако работим с 2 различни таблици, просто описваме WHERE table1.id = table2.id – т.е. правим връзка по начина, който сме правили с първични и външни ключове, просто не е графично, а с код. В случая, обаче, работим с една-

единствена таблица, затова, трябва да създадем нейно виртуално копие с псевдоним, защото иначе няма да може да се получи този изброяващ цикъл. Обикновено за псевдоним се взема първата буква на таблицата, в случая за peaks избрахме р. Ако има таблици, които започват с една и съща буква, се избират а, b, с ... Този метод изглежда по-сложен, но за 2 и повече таблици, е много по-практичен.

Последно, нека го запазим като ИЗГЛЕД на виртуална таблица – View:



VIEW = виртуална таблица

CREATE VIEW – и се използва по абсолютно същия начин като обикновената таблица, имената започват винаги с v_ ... и кода на заявката се задава като псевдоним с AS

CREATE OR REPLACE VIEW

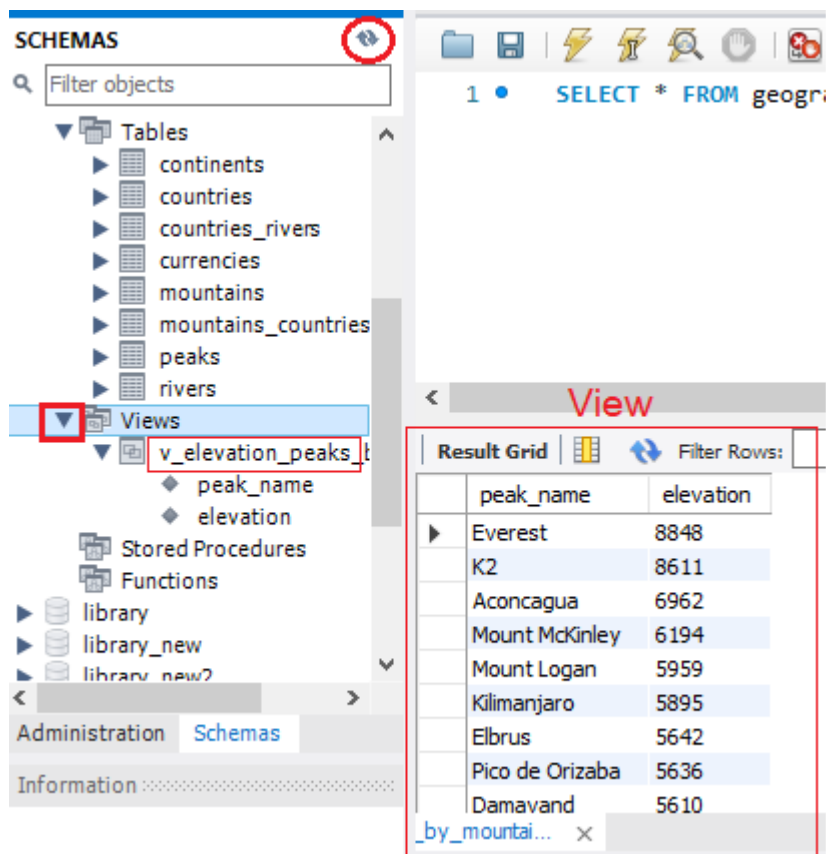
DROP VIEW

```
CREATE VIEW v_elevation_peaks_by_mountain AS
SELECT peak_name, elevation
FROM peaks AS p
WHERE elevation =
(
    SELECT elevation
    FROM peaks
    WHERE mountain_id = p.mountain_id -- цикъл
    ORDER BY elevation DESC
    LIMIT 1
)
ORDER BY elevation DESC;
```

Видимо не се случва нищо:

✓	88	12:38:46	CREATE VIEW v_elevation_peaks_by_mountain AS SELECT peak_name, elevation FRO...	0 row(s) affected
---	----	----------	---	-------------------

Но след Refresh на БД, виждаме новия изглед от падащото меню на Views:



Т.е. подзаявките са удобни, когато искаме да намерим някакъв резултат от дадена категория, например, всички имена и височини на върхове в България. За целта, таблица peaks не е достатъчна – от нея можем да вземем името на върха и неговата височина, но за да видим държавата, отиваме в другата таблица – междинната mountains_countries, в която „бръкваме“ благодарение на връзката, която сме създали чрез id. От там намираме кода на държавата, която ни интересува:

```
-- Намерете всички върхове в България и ги подредете по височина
SELECT peak_name, elevation
FROM peaks
WHERE mountain_id IN
(
    SELECT mountain_id
    FROM mountains_countries
    WHERE country_code = 'BG'
)
ORDER BY elevation DESC;
```

Също можем да припомним условно конкатениране, подобно на if-else в C# с (CASE WHEN условие THEN резултат за истина ELSE резултат за лъжа END) (ако ... то ... иначе ... край)

Ако служителят има средно име, да се изпише, ако не, да се остави интервал между името и фамилията. Другата задача е да се увеличат заплатите на всички служители с 10%.

```

USE company;
-- по-сложна конкатенация - ако съществува бащино име, да се изпише
CREATE VIEW v_employees_job_titles AS
SELECT
CONCAT(first_name, ' ', (case when middle_name IS NULL THEN '' ELSE middle_name END), ' ', last_name)
AS "Full Name", job_title
FROM employees;

-- вместо update за view
-- едновременно увеличение на заплатите на служителите с 10% от някои отдели
CREATE OR REPLACE VIEW v_employees_salary AS
SELECT salary*1.1 as salary_new
FROM employees
WHERE department_id IN (1, 2, 4, 11);
-- DROP VIEW - имената започват винаги с v_

```

Можем да „залепим“ резултатите от 2 подзаявки една под друга с клаузата UNION.

Важното е, че:

- броят на колоните от двете заявки трябва да е еднакъв (не може едната заявка да има повече или по-малко колони)
- псевдонимите с имената на колоните за резултата се задават в първата заявка
- допустимо е да има разминаване в типа на колоните (например, първо да се изреди в една колона числов тип int от първата заявка, а после да се продължи с varchar от другата заявка или обратно).

Нека изредим имената и дължините на 3 от най-високите върхове и 3 от най-дългите реки:

```

-- UNION - 3-те най-дълги реки и 3-те най-високи върха
(SELECT river_name AS Name, length AS Meters
FROM rivers
ORDER BY length DESC
LIMIT 3)
UNION
(SELECT peak_name, elevation
FROM peaks
ORDER BY elevation DESC
LIMIT 3);
-- ORDER BY name ASC;

```

Скобите за двете подзаявки от двете страни на UNION имат чисто декоративен характер. Кодът ще тръгне и без тях. Ако има подреждане на крайния резултат, той, както винаги, се прави на последно място извън последната заявка.

Резултатът е в общо 6 реда:

Name	Meters	псевдоними
Nile	6650	
Amazon	6400	rivers
Yangtze	6300	
Everest	8848	
K2	8611	peaks
Kangchenjunga	8586	

UNION залепва двете заявки една ПОД друга, докато JOIN една ДО друга.
UNION не е толкова практичен, колкото JOIN.

JOIN залепва 2 таблици една ДО друга, като първо изреждаме колоните, които искаме от двете таблици, след това изписваме FROM table1 JOIN table2 ON table1.id = table2.id, т.е. указваме коя е връзката, чрез ключовете, които сме правили. Ако не е изписан конкретен JOIN, по подразбиране е INNER.

```
-- JOIN - Името на върховете и планините, в които се намират
SELECT peak_name AS Peak, mountain_range AS Mountain
FROM peaks
JOIN -- INNER JOIN
mountains
ON peaks.mountain_id = mountains.id;
```

Вместо ON, може да се напише познатия WHERE, кодът ще тръгне, но не е прието за правилно.
WHERE = условие
ON = връзка.

Резултат 48 реда:

table1	table2
Peak	Mountain
Aconcagua	Andes
Botev	Balkan Mountains
Carstensen Pyramid	The Sudirman Range
Damavand	Alborz
Dykh-Tau	Caucasus
Elbrus	Caucasus
Everest	Himalayas

Последно: LIMIT дава много по-голяма гъвкавост от агрегатните функции, включително за пропускане / прескачане на резултат, например, ако искаме не просто най-дългата река Нил, а 3-те най-дълги реки (най-високи заплати) или обратното (най-къси реки, най-ниски заплати):

```

130 • SELECT river_name AS Name, length AS Meters
131 FROM rivers
132 ORDER BY length DESC
133 LIMIT 3;

```

Result Grid		Filter Rows:	Export:	Wrap Cell Content:
Name	Meters			
Nile	6650	най-дълга река ↓		
Amazon	6400			
Yangtze	6300			

OFFSET дава възможност за прескачане на резултат, като записът е в LIMIT:

```

-- OFFSET - Пропускане
-- Втората и третата най-дълги реки:
SELECT river_name AS Name, length AS Meters
FROM rivers
ORDER BY length DESC
-- LIMIT 3;      ЕКВИВАЛЕНТИ
-- LIMIT 1, 2;  -- пропуска 1, взема 2
LIMIT 2 OFFSET 1; -- взема 2, пропуска 1

```

Река Нил вече не присъства в отговора, прескочена е:

Name	Meters
Amazon	6400
Yangtze	6300